

trent-limited /ai-retail-vision-engine

Enterprise-grade AI Visual Search Engine & Dynamic Price Optimization System tailored for fashion brands (Westside & Zudio).

Python 64.2%

CUDA / C++ 21.8%

TypeScript 14.0%

★ 1,482 Stars | 294 Forks

18.4 ms

P95 VECTOR SEARCH LATENCY

142,500

INDEXED VISUAL SKUS

+14.2%

INVENTORY TURNOVER LIFT

⚡ System Architecture & Key Modules

1. Visual Search Vector Pipeline

Utilizes fine-tuned CLIP / Swin Transformer backbones to project apparel imagery into a 512-dimensional metric space indexed via Milvus / FAISS.

PyTorch

Milvus Vector DB

TensorRT

- Sub-20ms nearest-neighbor lookup across millions of visual embeddings.
- Automatic background removal and garment attribute tagging.
- Cross-modal text-to-image and image-to-image search integration.

2. Dynamic Pricing & Markdown Engine

Reinforcement Learning (PPO) & Gradient Boosted Demand Models optimize prices based on inventory velocity, competitor tracking, and seasonality.

XGBoost

RLlib / PPO

FastAPI

- Dynamic price elasticities calculation per SKU.
- Margin-preserving markdown schedules for end-of-season inventory.
- Real-time telemetry stream for price elasticity monitoring.

📐 Core Formulation & Optimization Objective

The dynamic price optimization model balances inventory clearance velocity against profit margin maximization under elasticity bounds:

$$P^* = \underset{p \in [C(1+\gamma), MRP]}{\operatorname{argmax}} \left[(p - C) \cdot D(p, I, S, P_{comp}) - \lambda \cdot \max(0, I - D(p) \cdot \Delta t) \right]$$

Where P^* is optimal price, C is cost, $D(\dots)$ is predicted demand function, I is current inventory level, S is seasonality index, and λ represents deadstock penalty factor.

FastAPI High-Throughput Endpoint Snippet

```
from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel
import numpy as np

app = FastAPI(title="Trent AI Engine API", version="1.0.0")

class PricingRequest(BaseModel):
    sku_id: str
    inventory_level: int
    competitor_price: float
    seasonality_index: float = 0.5

@app.post("/api/v1/pricing/optimize")
async def optimize_price(req: PricingRequest):
    """Compute optimal markdown based on elasticity curve"""
    optimal_mrp, margin_delta, rec = evaluate_pricing_policy(
        req.sku_id, req.inventory_level, req.competitor_price, req.seasonality_index
    )
    return {
        "status": "success",
        "sku_id": req.sku_id,
        "optimal_mrp": optimal_mrp,
        "margin_delta_pct": margin_delta,
        "recommendation": rec
    }
```

 Benchmark Performance across Retail Catalog

Brand / Division	SKU Count	Embedding Latency (P99)	Demand Precision (F1)	Markdown Conversion
Westside Womenswear	48,200	17.2 ms	0.92	+18.5%
Westside Menswear & Footwear	34,100	16.8 ms	0.89	+12.1%
Zudio Everyday Apparel	60,200	19.1 ms	0.94	+22.4%